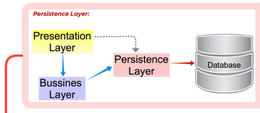
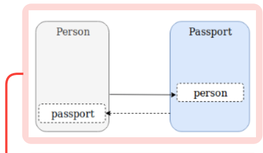
Java Web Services - Session 02

① Tổng quan về JPA



Entity

- Entity là đối tượng đại diện cho dữ liệu ở trong ứng dụng
- Entity thường là POJO (Plain Old Java Object)
- Entity sẽ được ánh xạ (mapping) tới một bảng trong CSDL
- Trong JPA, một entity cần tuân thủ:
 1. Được gắn với annotation javax.persistence.Entity
 2. Có một constructor là public và không có tham số
 3. Không được khai báo final
 4. Các biến đối tượng cần được khai báo là private, protected hoặc ở mức package-private
- Các loại quan hệ trong JPA
 - Quan hệ 1-1: @OneToOne



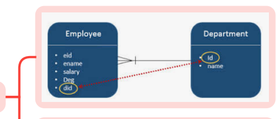

```
@Entity
public final class Person {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToOne(mappedBy = "person")
    private Passport passport;

    //constructor
    //getters
}
```

 - Quan hệ 1-n: @OneToMany



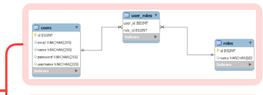

```
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "department_id", referencedColumnName = "depId")
private Set<Employee> employees;
```

 - Quan hệ n-1: @ManyToOne



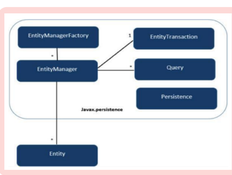

```
@ManyToOne
@JoinColumn(name = "department_id", referencedColumnName = "depId")
private Department department;
```

 - Quan hệ n-n: @ManyToMany




```
@ManyToMany
@JoinTable(
    name = "user_role",
    joinColumns = @JoinColumn(name = "role_id"),
    inverseJoinColumns = @JoinColumn(name = "user_id")
)
private Set<User> users;
```

PersistenceContext và EntityManager



- PersistenceContext là tập các thể hiện của entity được quản lý, tồn tại trong một kho dữ liệu.
- Interface EntityManager:
 - Khai báo các phương thức để tương tác với persistence context
 - Tạo hoặc xoá các thể hiện của entity
 - Tìm kiếm entity theo khoá chính
 - Thực thi các câu lệnh truy vấn lên entity

Quản lý các entity

- Container-managed Entity Manager

```
@PersistenceContext
EntityManager em;
```

- Application-managed Entity Manager

```
@PersistenceUnit
EntityManagerFactory emf;

EntityManager em = emf.createEntityManager();
```

② Các thao tác cơ bản trong JPA

1. Tìm 1 entity

```
@PersistenceContext  EntityManager em;

public void enterOrder(int custID, Order
newOrder) {
    Customer cust = em.find(Customer.class, custID);
    cust.getOrders().add(newOrder);
    newOrder.setCustomer(cust);
}
```

2. Lưu trữ 1 entity

```
@PersistenceContext
EntityManager em;
...
public LineItem createLineItem(Order order, Product product, int quantity) {
    LineItem li = new LineItem(order, product, quantity);
    order.getLineItems().add(li);
    em.persist(li);
    return li;
}
```

3. Xóa 1 entity

```
@PersistenceContext
EntityManager em;
...
public void removeOrder(Integer orderId) {
try {
  Order order = em.find(Order.class,orderId); em.remove(order);
}
```

4. Câu lệnh truy vấn động

```
- Phương thức createQuery() của lớp EntityManager giúp tạo các câu truy vấn động
(dynamic query)
- Ví dụ:
    public List findWithName(String name) {
        return em.createQuery(
            "SELECT c FROM Customer c WHERE c.name LIKE
            :custName")
            .setParameter("custName", name)
            .setMaxResults(10)
            .getResultList();
    }
```

5. Câu lệnh truy vấn động - NamedQuery
 - Khai báo Named Query
 - Phương thức createNamedQuery() của lớp EntityManager giúp tạo các câu truy vấn tĩnh (static query)
 - Ví dụ, khai báo Named Query:

```
@NamedQuery( name="findAllCustomersWithName",
query="SELECT c FROM Customer c WHERE c.name LIKE :custName"
)
```

 - Sử dụng Named Query

```
@PersistenceContext
public EntityManager em;
...
    customers = em.createNamedQuery("findAllCustomersWithName")
    .setParameter("custName", "Smith")
    .getResultList();
```

6. Câu lệnh truy vấn động - Named Parameter

```
public List findWithName(String name) {
  return em.createQuery(
      "SELECT c FROM Customer c WHERE c.name LIKE :custName")
      .setParameter("custName", name)
      .getResultList();
}
```

7. Câu lệnh truy vấn động - positional parameter

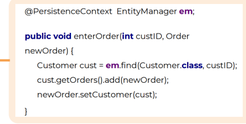
```
public List findWithName(String name) {
  return em.createQuery(
      "SELECT c FROM Customer c WHERE c.name LIKE ?")
      .setParameter(1, name)
      .getResultList();
}
```

③ Quản lý transaction trong JPA

```
@PersistenceContext EntityManagerFactory emf; EntityManager
em; @Resource UserTransaction utx;
...
    em = emf.createEntityManager();
    try {
        utx.begin(); em.persist(SomeEntit y); em.merge(AnotherEn tity);
        em.remove(ThirdEntit y); utx.commit();
    } catch (Exception e) {
        utx.rollback();
    }
```

④ Cấu hình

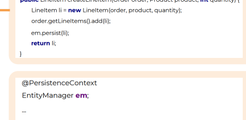
Spring Data JPA



Lựa chọn tầng truy xuất dữ liệu



Spring Data



Dependencies

```
dependencies {
    compile group: 'org.springframework.data', name: 'spring-data-jpa', version:
    '2.0.7.RELEASE'
    compile group: 'mysql', name: 'mysql-connector-java', version: '8.0.11'
    testCompile group: 'org.junit.jupiter', name: 'junit-jupiter-engine', version: '5.2.0'
}
```

DataSource

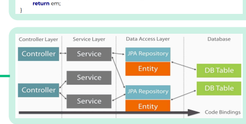
```
@Bean
    public DataSource dataSource(){
    DriverManagerDataSource dataSource =
    new DriverManagerDataSource();
    dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
    dataSource.setUrl("jdbc:mysql://localhost:3306/phone_store");
    dataSource.setUsername( "root" );
    dataSource.setPassword( "123456" );
    return dataSource;
    }
```

EntityManager

```
@Bean
@Qualifier(value = "entityManager")
public EntityManager entityManager(EntityManagerFactory entityManagerFactory) {
        return entityManagerFactory.createEntityManager();
}

@Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(){
    LocalContainerEntityManagerFactoryBean em =
        new LocalContainerEntityManagerFactoryBean();
    em.setDataSource(dataSource());
    em.setPackagesToScan(new String[] {"ra.phonestore.model" });
    JpaVendorAdapter vendorAdapter = new HibernateJpaVendorAdapter();
    em.setJpaVendorAdapter(vendorAdapter);
    em.setJpaProperties(additionalProperties());
    return em;
}
```

Kiến trúc phân tầng



Presented with xmind